Development Documentation

AI-Based Healthcare Anomaly Detection Platform

1. Project Overview

The **AI-Driven Healthcare Anomaly Detection System** is a real-time monitoring platform designed to detect abnormal patterns in patient vital signs using machine learning techniques. The primary goal of the system is to assist healthcare professionals by identifying potential health risks early and generating alerts when abnormal conditions are detected.

Traditional healthcare monitoring systems rely on predefined thresholds and manual monitoring, which may fail to detect subtle abnormal patterns in patient data. This project overcomes these limitations by using **machine learning models and real-time data streaming technologies** to continuously analyze physiological data.

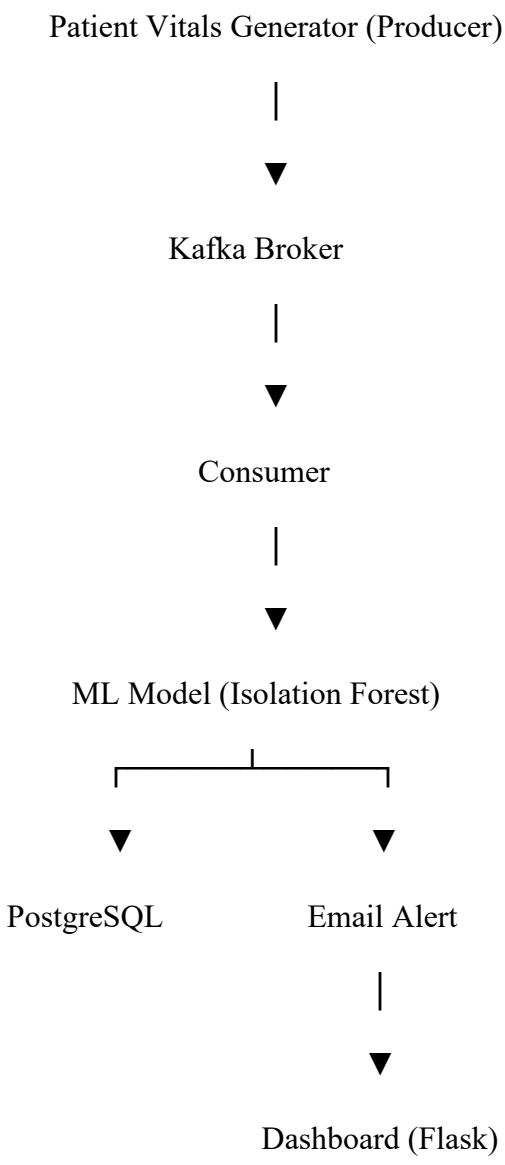
The system collects patient vitals such as **heart rate, blood pressure, body temperature, and oxygen saturation** and streams the data through **Apache Kafka**. The data is processed by an **Isolation Forest anomaly detection model**, which identifies abnormal patterns in the incoming data. Detected anomalies are stored in a **PostgreSQL database**, and critical cases automatically trigger **email alerts** to healthcare administrators.

The platform demonstrates the integration of **machine learning, real-time streaming systems, backend APIs, and database technologies** to create an intelligent healthcare monitoring system.

2. System Architecture

Overall System Architecture

The system follows a **streaming architecture** where patient vital data is continuously generated, processed, and analyzed in real time.



Components Explanation

Producer

The **Producer Module** is responsible for generating or collecting patient vital signs such as heart rate, temperature, oxygen saturation, and blood pressure. These vitals are formatted into structured messages and sent to a Kafka topic for real-time processing.

The producer simulates patient health data and streams it continuously to the Kafka broker.

Kafka Broker

The **Apache Kafka Broker** acts as a real-time streaming platform that receives patient data from the producer and stores it temporarily in topics. It ensures reliable data transmission between producers and consumers.

Kafka allows the system to handle high-volume streaming data efficiently and ensures that messages are delivered in the correct order.

Consumer

The **Consumer Module** reads the patient vital data from the Kafka topic. Once the data is received, it preprocesses the information and forwards it to the machine learning model for anomaly detection.

The consumer acts as the bridge between the streaming data pipeline and the machine learning system.

Machine Learning Model

The **Isolation Forest Model** analyzes the incoming patient vital data and calculates anomaly scores. If the system detects abnormal patterns in the vitals, it classifies the data as an anomaly.

The model helps identify unusual physiological patterns that may indicate potential health risks.

Database

All detected anomalies and patient data are stored in a **PostgreSQL database**. The database maintains records such as patient ID, vital values, anomaly status, severity level, and timestamps.

This stored data can later be used for analysis, reporting, and visualization in the dashboard.

Email Alert System

When the machine learning model detects a **high-severity anomaly**, the system automatically triggers an **email alert** using the Mail trap email service.

These alerts notify healthcare administrators or doctors so that they can take immediate action if required.

3. Technologies Used

The following technologies were used during the development of the AI-Based Healthcare Anomaly Detection System.

Technology	Purpose
Python	Core programming language used for system development
Apache Kafka	Real-time data streaming platform
PostgreSQL	Database used to store anomaly logs
scikit-learn	Machine learning library used for anomaly detection
Mailtrap	Email service used to simulate alert notifications
Pandas	Data processing and manipulation
Flask	Web framework used to build the monitoring dashboard
JSON	Data format used for transmitting patient vitals

These technologies collectively enable real-time data ingestion, machine learning processing, and visualization.

4. Dataset Description

Dataset Name

Patient Vitals Monitoring Dataset

Dataset Source

The dataset used in this project is **synthetically generated** to simulate real-time patient physiological data for testing the anomaly detection system.

Features

The dataset contains the following attributes:

Feature	Description
patient_id	Unique identifier for each patient

Feature	Description
heart_rate	Patient heart rate measured in beats per minute
blood_pressure	Patient blood pressure reading
temperature	Body temperature of the patient
spo2	Oxygen saturation level
timestamp	Time at which the vitals were recorded

Data Format

The data is transmitted in **JSON format** through the Kafka streaming pipeline.

Example JSON message:


```
{
  "patient_id": 101,
  "heart_rate": 82,
  "blood_pressure": 120,
  "temperature": 98.6,
  "spo2": 97
}
```


5. Machine Learning Model

Model Name

Isolation Forest

Purpose of the Model

The **Isolation Forest algorithm** is used for anomaly detection. It identifies unusual patterns in patient vital signs that deviate significantly from normal behavior.

Unlike traditional classification models, Isolation Forest is specifically designed to detect rare or abnormal observations in large datasets.

Training Process

1. The historical patient vital data is collected and preprocessed.
2. Relevant features such as heart rate, blood pressure, temperature, and oxygen saturation are selected.
3. The Isolation Forest model is trained using the **scikit-learn library**.
4. The trained model learns normal patterns of patient vitals.
5. During real-time operation, the model evaluates incoming data and assigns an anomaly score.

Input Features

The model takes the following features as input:

- Heart Rate
- Blood Pressure
- Temperature
- Oxygen Saturation (SpO_2)

Model Output

The model classifies the incoming data into two categories:

Output	Meaning
Normal	Patient vitals are within acceptable range
Anomaly	Abnormal vital patterns detected

Based on the anomaly score, the system also assigns a **severity level** such as LOW, MEDIUM, or HIGH.

6. Module Development

The system is divided into multiple modules for efficient development and maintenance.

Producer Module

The **Producer Module** generates or collects patient vital signs and streams them into a Kafka topic.

Functions of Producer Module:

- Generates patient vitals data
- Converts the data into JSON format
- Sends the data to Kafka topic

Consumer Module

The **Consumer Module** receives streaming data from Kafka and processes it using the machine learning model.

Functions of Consumer Module:

- Reads data from Kafka topic
- Preprocesses patient vital data
- Runs anomaly detection model
- Classifies anomalies

Database Module

The **Database Module** stores detected anomalies in PostgreSQL for future analysis.

Functions:

- Stores anomaly records
- Maintains patient vital history
- Enables querying and reporting

Alert Module

The **Alert Module** sends automated email notifications when a severe anomaly is detected.

Functions:

- Detects high-severity anomaly events
- Generates alert messages
- Sends email notifications using Mail trap

7. Database Design

The PostgreSQL database contains a table used to store anomaly detection logs.

Table Name

```
Anomaly Logs
```

Column Name	Data Type	Description
id	serial	Unique identifier
patient_id	integer	Patient ID
heart_rate	integer	Heart rate value
blood_pressure	integer	Blood pressure value
temperature	float	Body temperature
severity	varchar	Anomaly severity level
timestamp	timestamp	Record creation time

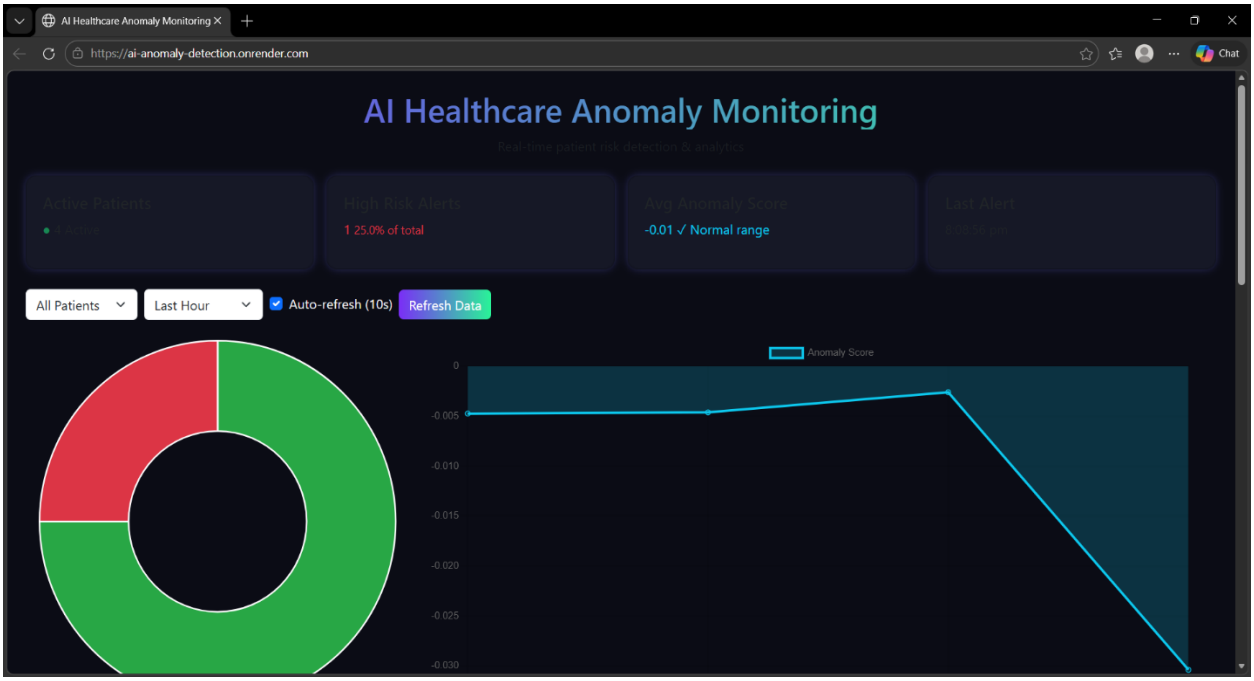
This table stores all anomaly detection results generated by the system.

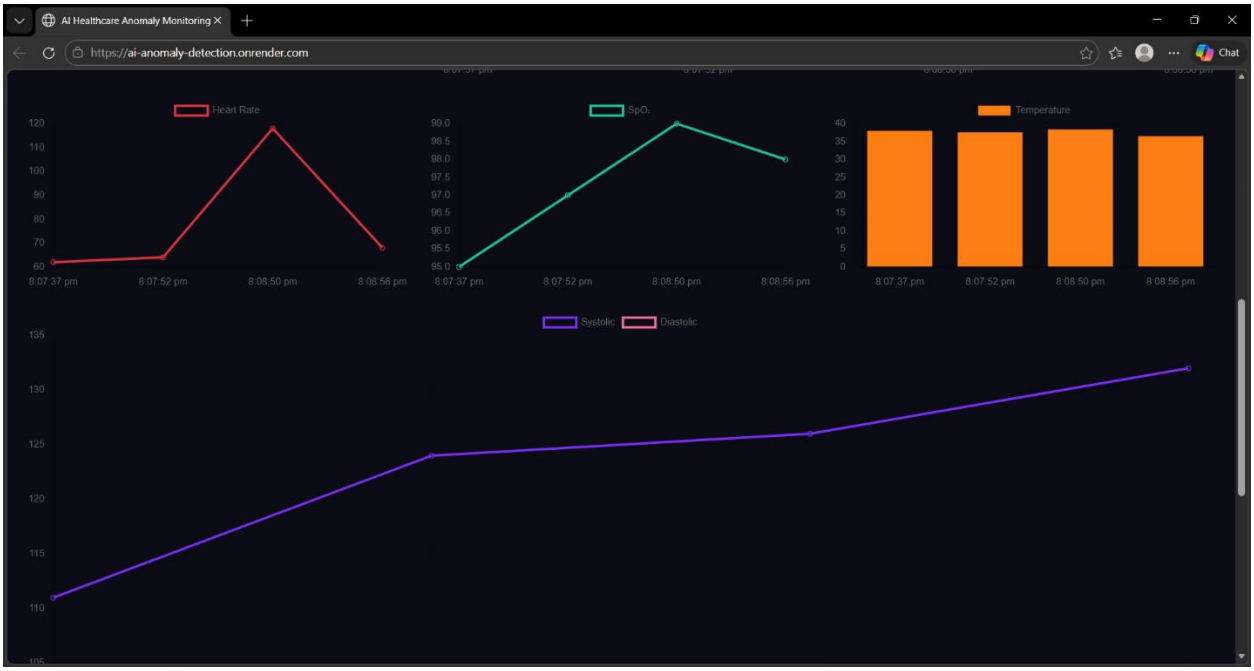
8. Screenshots

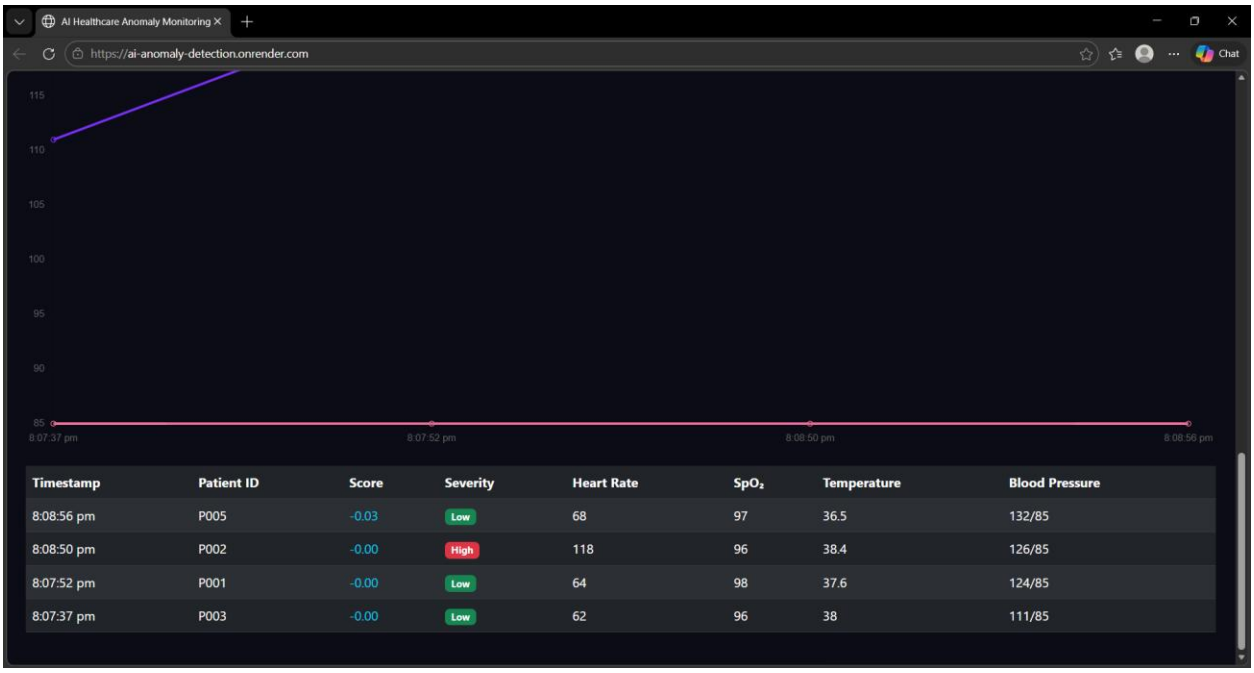
This section presents screenshots demonstrating the implementation and working of the AI-Based Healthcare Anomaly Detection System.

8.1 Dashboard Visualization

The Flask-based dashboard provides a real-time visualization of patient vitals and detected anomalies. Healthcare administrators can monitor patient health status through this interface.







8.2 Kafka Broker Running

This screenshot shows the producer generating patient vital data and sending it to the Kafka topic.

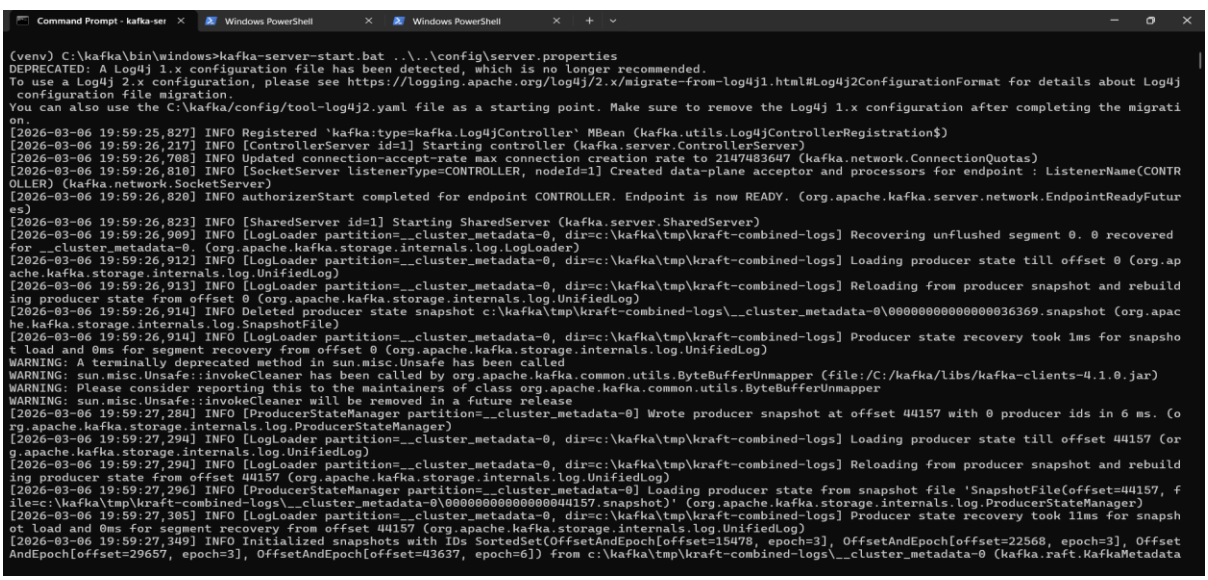


Figure 8.2 : Kafka broker

8.3 Kafka Producer Sending Patient Data Request

This screenshot shows the producer sending patient data request from Kafka and passing it to the machine learning model for anomaly detection.

```
Command Prompt - kafka-serv   Windows PowerShell   Windows PowerShell
(venv) PS C:\Projects\Ai-anomaly_detection\kafka> python kafka_producer.py
Kafka Producer started...

Produced: {'patient_id': 3, 'heart_rate': 73, 'blood_pressure': 147, 'temperature': 37.3, 'timestamp': 1772807815.5792744}
Produced: {'patient_id': 4, 'heart_rate': 66, 'blood_pressure': 148, 'temperature': 38.2, 'timestamp': 1772807816.7041829}
Produced: {'patient_id': 3, 'heart_rate': 67, 'blood_pressure': 135, 'temperature': 38.2, 'timestamp': 1772807817.7095401}
Produced: {'patient_id': 2, 'heart_rate': 71, 'blood_pressure': 132, 'temperature': 37.5, 'timestamp': 1772807818.7170815}
Produced: {'patient_id': 3, 'heart_rate': 85, 'blood_pressure': 122, 'temperature': 37.2, 'timestamp': 1772807819.7364595}
Produced: {'patient_id': 2, 'heart_rate': 86, 'blood_pressure': 117, 'temperature': 37.1, 'timestamp': 1772807820.743498}
Produced: {'patient_id': 5, 'heart_rate': 103, 'blood_pressure': 124, 'temperature': 37.4, 'timestamp': 1772807821.7568352}
Produced: {'patient_id': 1, 'heart_rate': 98, 'blood_pressure': 147, 'temperature': 37.0, 'timestamp': 1772807822.765678}
Produced: {'patient_id': 3, 'heart_rate': 73, 'blood_pressure': 135, 'temperature': 38.9, 'timestamp': 1772807823.7742362}
Produced: {'patient_id': 2, 'heart_rate': 110, 'blood_pressure': 149, 'temperature': 38.7, 'timestamp': 1772807824.7889225}
Produced: {'patient_id': 1, 'heart_rate': 110, 'blood_pressure': 111, 'temperature': 38.4, 'timestamp': 1772807825.7971382}
Produced: {'patient_id': 1, 'heart_rate': 120, 'blood_pressure': 146, 'temperature': 37.3, 'timestamp': 1772807826.805269}
Produced: {'patient_id': 2, 'heart_rate': 77, 'blood_pressure': 144, 'temperature': 36.7, 'timestamp': 1772807827.8067153}
Produced: {'patient_id': 4, 'heart_rate': 92, 'blood_pressure': 142, 'temperature': 38.7, 'timestamp': 1772807828.8200867}
Produced: {'patient_id': 5, 'heart_rate': 94, 'blood_pressure': 122, 'temperature': 37.7, 'timestamp': 1772807829.8294582}
Produced: {'patient_id': 5, 'heart_rate': 118, 'blood_pressure': 135, 'temperature': 37.7, 'timestamp': 1772807830.8344915}
Produced: {'patient_id': 2, 'heart_rate': 78, 'blood_pressure': 113, 'temperature': 37.6, 'timestamp': 1772807831.8421998}
Produced: {'patient_id': 4, 'heart_rate': 119, 'blood_pressure': 140, 'temperature': 37.1, 'timestamp': 1772807832.8482482}
Produced: {'patient_id': 5, 'heart_rate': 88, 'blood_pressure': 139, 'temperature': 37.5, 'timestamp': 1772807833.8589904}
Produced: {'patient_id': 4, 'heart_rate': 107, 'blood_pressure': 120, 'temperature': 38.7, 'timestamp': 1772807834.867403}
Produced: {'patient_id': 3, 'heart_rate': 64, 'blood_pressure': 112, 'temperature': 36.5, 'timestamp': 1772807835.8769376}
Produced: {'patient_id': 5, 'heart_rate': 77, 'blood_pressure': 145, 'temperature': 38.7, 'timestamp': 1772807836.885631}
Produced: {'patient_id': 1, 'heart_rate': 93, 'blood_pressure': 120, 'temperature': 38.7, 'timestamp': 1772807837.8933632}
Produced: {'patient_id': 1, 'heart_rate': 81, 'blood_pressure': 134, 'temperature': 38.6, 'timestamp': 1772807838.9049048}
Produced: {'patient_id': 3, 'heart_rate': 113, 'blood_pressure': 125, 'temperature': 37.5, 'timestamp': 1772807839.9149318}
Produced: {'patient_id': 4, 'heart_rate': 76, 'blood_pressure': 143, 'temperature': 38.9, 'timestamp': 1772807840.9298415}
Produced: {'patient_id': 3, 'heart_rate': 106, 'blood_pressure': 133, 'temperature': 37.4, 'timestamp': 1772807841.9472609}
Produced: {'patient_id': 3, 'heart_rate': 75, 'blood_pressure': 142, 'temperature': 36.8, 'timestamp': 1772807842.9479978}
Produced: {'patient_id': 2, 'heart_rate': 95, 'blood_pressure': 124, 'temperature': 37.5, 'timestamp': 1772807843.956848}
Produced: {'patient_id': 4, 'heart_rate': 90, 'blood_pressure': 128, 'temperature': 37.1, 'timestamp': 1772807844.969526}
Produced: {'patient_id': 3, 'heart_rate': 100, 'blood_pressure': 145, 'temperature': 36.7, 'timestamp': 1772807845.9743116}
Produced: {'patient_id': 3, 'heart_rate': 79, 'blood_pressure': 121, 'temperature': 37.9, 'timestamp': 1772807846.9843228}
Produced: {'patient_id': 4, 'heart_rate': 64, 'blood_pressure': 128, 'temperature': 37.6, 'timestamp': 1772807847.9975731}
Produced: {'patient_id': 3, 'heart_rate': 112, 'blood_pressure': 144, 'temperature': 38.3, 'timestamp': 1772807849.0101883}
Produced: {'patient_id': 1, 'heart_rate': 88, 'blood_pressure': 120, 'temperature': 38.7, 'timestamp': 1772807850.0199816}
Produced: {'patient_id': 1, 'heart_rate': 87, 'blood_pressure': 147, 'temperature': 37.2, 'timestamp': 1772807851.0327559}
Produced: {'patient_id': 3, 'heart_rate': 108, 'blood_pressure': 121, 'temperature': 38.3, 'timestamp': 1772807852.0410147}
Produced: {'patient_id': 4, 'heart_rate': 106, 'blood_pressure': 139, 'temperature': 38.9, 'timestamp': 1772807853.0558643}
```

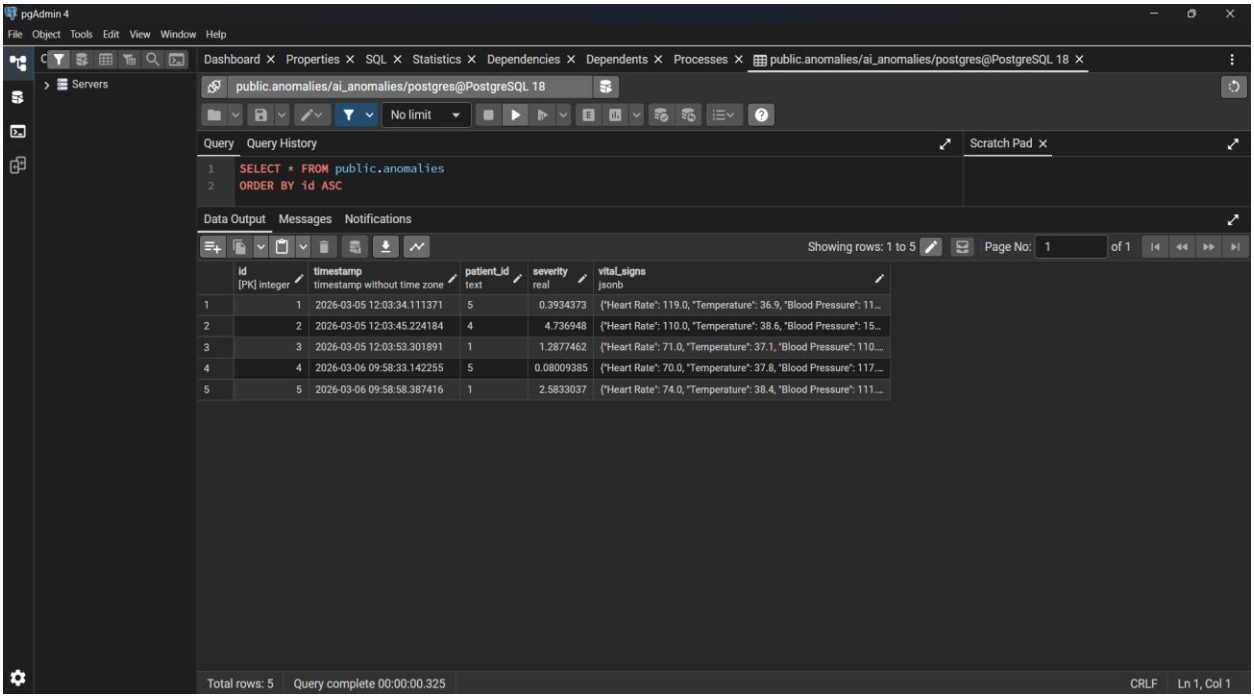
8.4 Kafka Consumer Processing Data

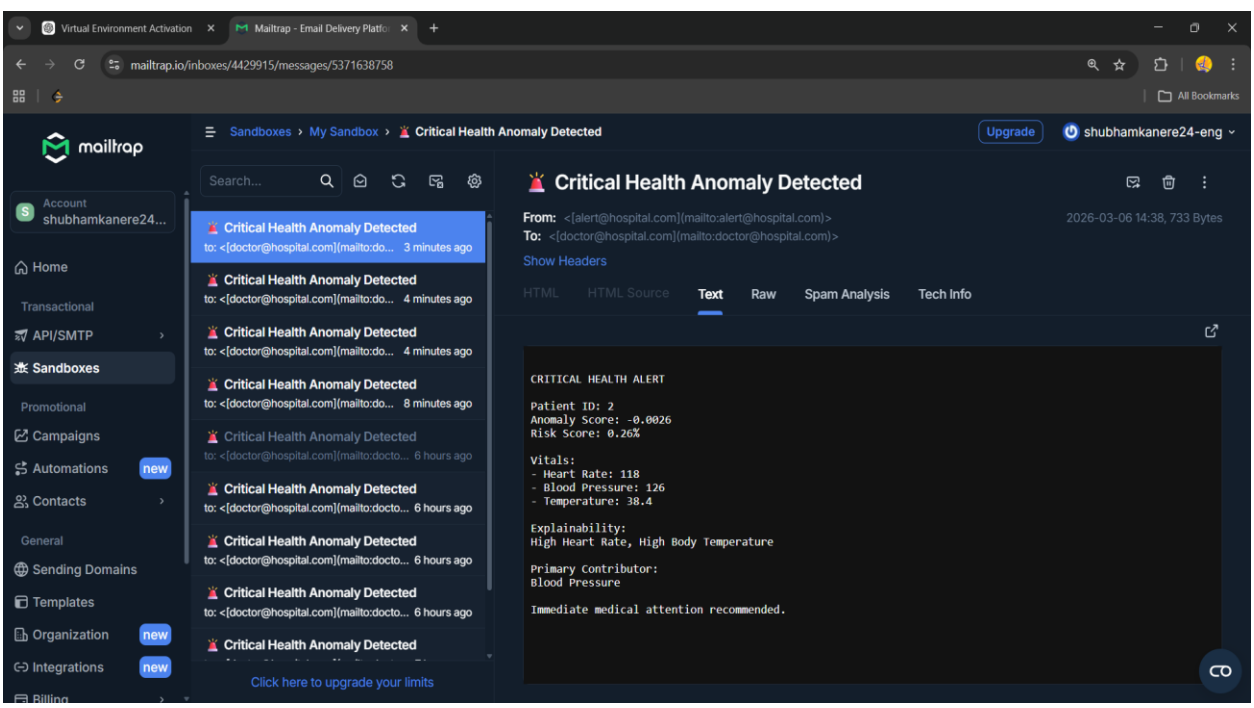
This screenshot shows the consumer receiving data from Kafka and passing it to the machine learning model for anomaly detection.

```
Command Prompt - kafka-serv   Windows PowerShell   Windows PowerShell
✅ Patient 1 is Normal.

Received data for Patient 2: {'patient_id': 2, 'heart_rate': 73, 'blood_pressure': 145, 'temperature': 38.5, 'timestamp': 1772807929.8211887}
C:\Projects\Ai-anomaly_detection\venv\lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X does not have valid feature names, but IsolationFor
est was fitted with feature names
  warnings.warn(
C:\Projects\Ai-anomaly_detection\venv\lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X does not have valid feature names, but IsolationFor
est was fitted with feature names
  warnings.warn(
✅ Patient 2 is Normal.

Received data for Patient 2: {'patient_id': 2, 'heart_rate': 118, 'blood_pressure': 126, 'temperature': 38.4, 'timestamp': 1772807930.8378167}
C:\Projects\Ai-anomaly_detection\venv\lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X does not have valid feature names, but IsolationFor
est was fitted with feature names
  warnings.warn(
C:\Projects\Ai-anomaly_detection\venv\lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X does not have valid feature names, but IsolationFor
est was fitted with feature names
  warnings.warn(
📧 Email alert sent successfully
⚠️PostgreSQL cursor not available, skipping insert.

Received data for Patient 2: {'patient_id': 2, 'heart_rate': 106, 'blood_pressure': 137, 'temperature': 37.0, 'timestamp': 1772807931.8504746}
C:\Projects\Ai-anomaly_detection\venv\lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X does not have valid feature names, but IsolationFor
est was fitted with feature names
  warnings.warn(
C:\Projects\Ai-anomaly_detection\venv\lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X does not have valid feature names, but IsolationFor
est was fitted with feature names
  warnings.warn(
✅ Patient 2 is Normal.

Received data for Patient 1: {'patient_id': 1, 'heart_rate': 103, 'blood_pressure': 141, 'temperature': 38.0, 'timestamp': 1772807932.8643866}
C:\Projects\Ai-anomaly_detection\venv\lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X does not have valid feature names, but IsolationFor
est was fitted with feature names
  warnings.warn(
C:\Projects\Ai-anomaly_detection\venv\lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X does not have valid feature names, but IsolationFor
est was fitted with feature names
  warnings.warn(
✅ Patient 1 is Normal.

Received data for Patient 5: {'patient_id': 5, 'heart_rate': 114, 'blood_pressure': 129, 'temperature': 37.8, 'timestamp': 1772807933.8768585}
C:\Projects\Ai-anomaly_detection\venv\lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X does not have valid feature names, but IsolationFor
```

8.5 Database Records in PostgreSQL

This screenshot shows anomaly records stored in the PostgreSQL database.



8.7 Email Alert Notification

This screenshot shows the email alert sent when a high-severity anomaly is detected.